# Smart Voicemail Router

## دليل الأدوار وسير العمل

**رقم المستند:** SVR-TEAM-04

**الإصدار:** 1.0

**آخر تحديث:** 12 نوفمبر 2026

**المستند المرافق:** SVR-TEAM-03 Team Allocation v1.3 (الأدوار والمراحل باختصار)

## الدليل ده عن إيه

الدليل ده بيشرح لكل واحد في الفريق دوره بالظبط، يعمل إيه، ويعمله إزاي، من أول مرة تنسخ فيها المشروع على جهازك لحد ما النظام يخلص. اقرأ قسم دورك كله. واقرأ قسم سير العمل الموازي وقسم الاعتماديات، عشان تعرف مين محتاج منك إيه وامتى. مفيش حاجة هنا مجرد لقب. كل دور ليه شغل حقيقي، ومخرجات واضحة، وطريقة واضحة تعرف بيها إنك خلصت.

المشروع بيحوّل رسالة صوتية واحدة من عميل باللهجة المصرية لـcallback ticket واضحة ومفسّرة. dataset خلصت. وmodel selection محسوم. اللي فاضل هو نبني models، ونربط النظام، ونقيّمه، وبعدين retrain مرة واحدة على الـ labels المراجعة.

## الصورة الكبيرة

النظام بيمشي في أربع خطوات بالترتيب:

1. **الصوت بيدخل.** رسالة صوتية بتوصل. بنفحصها، ونوحّدها لملف WAV بتردد 16 كيلوهرتز وقناة واحدة، وبعدين نحوّلها لنص باستخدام Whisper large-v3.
2. **الفهم.** model بيقرأ النص وبيتوقع أربع حاجات: فئة الشكوى، ودرجة الاستعجال، وحالة العميل، واحتمال فقدان العميل. وaudio model ممكن يضيف دليل من الصوت نفسه.

**ملحوظة عن البيانات:** الرسائل الصوتية دي تمثيلية. الناس سجلوها عشان نجمع البيانات، مش عملاء حقيقيين، والمشاعر اللي فيها متصطنعة. حتى الإنسان نفسه مش بيقدر يحدد الـ emotion (زي mad أو calm أو غيرها) من الصوت لوحده بشكل موثوق. عشان كده الـ transcription والكلمات هما الـ main signal: الـ category والـ urgency والـ customer state كلهم بيتحددوا من الكلمات. الـ audio emotion model دي تجربة، ولازم يثبت إنه بيضيف signal فوق الـ text لوحده، ولو مضافش حاجة بنستغنى عنه.

1. **القرار.** منطق الفرز بيحوّل الحقول الأربعة لـqueue priority وcallback ticket مفسّرة.
2. **التسليم.** الـticket بتروح للموظف مع السبب، والتشغيل كله بيتقاس من ناحية accuracy and speed.

dataset هي SVR Voicemail Corpus: عدد 6,522 رسالة صوتية، وعدد 5,751 صف تدريب نظيف، متقسمة train 4,031، و validation 1,136، و test 584، seed 42. الـ labels جاية من model، والفريق بيراجعها كل يوم لمدة شهر تقريبا. التدريب ماشي بالتوازي مع المراجعة على الـ labels الحالية. ولما المراجعة تخلص، الـ labels بتتصحح وmodels بتتدرب من تاني. retrain دي هي المرحلة الرابعة.

الشغل ماشي في أربع مراحل:

- **المرحلة 1: الأساسيات.** كل دور بيبني الجزء بتاعه على fixtures وmocks. محدش بيستنى حد.
- **المرحلة 2: core models.** text classifier وaudio model بيتدربوا وبيتعمل لهم validation. والفائز بيتحدد بمتوسط macro-F1 على validation set.
- **المرحلة 3: evaluation والربط.** test set بتتقيّم مرة واحدة. ومنطق الفرز بيخلص على مخرجات real models. وpipeline الكامل بيتربط وdemo بيتبني.
- **المرحلة 4: retrain على الـ labels المراجعة.** بعد ما المراجعة تخلص، الـ labels بتتصحح وtraining notebooks بتتشغل من تاني. مفيش تغيير في architecture ولا في hyperparameters.

## البداية: من النسخ لأول تشغيل

كل عضو بيمشي على الخطوات دي بالترتيب قبل ما يبدأ شغل المرحلة 1. ولو أي خطوة فشلت، وقف واسأل حازم. ومتتحايلش على إعداد فاشل.

### الخطوة 1: ثبّت الأدوات

ثبّت Git وPython 3.11 أو أحدث على جهازك. وتأكد إن الاتنين شغالين:

```
git --version python3 --version
```

### الخطوة 2: انسخ المستودع

```
git clone https://github.com/hazemelerefey/smart-voicemail-router.git svr cd svr
```

### الخطوة 3: روح على فرع دورك

الكود بتاعك عايش على فرع دورك. روح عليه:

```
git checkout team/<your-name>-<your-role>
```

اسم فرعك موجود في جدول الأدوار في القسم 2 من SVR-TEAM-03 وفي قسم دورك تحت. فمثلا سيرا بتنفذ `git checkout team/sira-audio-speech`.

## الخطوة 4: هات البيانات

dataset عايشة على الفرع `smart-voicemail-router` ، منفصلة عن فروع الكود، عشان الصوت حجمه كبير. هاتها جنب الكود باستخدام worktree عشان يكون عندك الاتنين مع بعض:

```
git worktree add ../svr-data smart-voicemail-router
```

دلوقتي بقى عندك مجلدين. المجلد `svr/` فيه فرع دورك والكود. والمجلد `svr-data/` فيه dataset: الصوت في `dataset/train` و `dataset/val` و `dataset/test` ، والدفاتر في `notebooks/` ، وtraining tables في `final/` . مجلد البيانات ده مرجع ليك. اقرأ منه. ومتعدّلش الصوت ولا الجداول اللي فيه. والكود بيشاور على مجلد البيانات بمسار، زي `../svr-data/dataset` .

## الخطوة 5: جهّز بيئة Python

اعمل بيئة افتراضية وفعّلها:

```
python3 -m venv .venv source .venv/bin/activate
```

ثبّت الحزم اللي دورك محتاجها. الحزم الأساسية للكل هي pandas وnumpy وscikit-learn. وكل قسم دور تحت بيسرد الحزم الزيادة للدور ده. وتأكد إن التثبيت شغال باستيراد الحزمة الرئيسية لدورك، زي `python -c "import transformers"` للمى وسيرا.

## الخطوة 6: اتأكد من البيانات

نفّذ الفحوصات دي قبل ما تبدأ شغل. ولازم كلها تنجح:

1. training tables موجودة في `svr-data/final/` : وهي clean_training_train.csv وclean_training_val.csv وclean_training_test.csv.
2. عدد الصفوف هو 4,031 train، و 1,136 validation، و 584 test، والمجموع 5,751.
3. مجلدات الصوت فيها العدد نفسه: `dataset/train` فيه 4,031 ملف، و `dataset/val` فيه 1,136 ملف، و `dataset/test` فيه 584 ملف.
4. كل ملف صوتي هو WAV بتردد 16 كيلوهرتز وقناة واحدة وبيشتغل.

ولو أي عدد غلط، وقف وقول لحازم. ومتدرّبش على dataset متأكدتش منها.

## الخطوة 7: ابدأ المرحلة 1

اقرأ قسم دورك اللي تحت. ونفّذ شغل المرحلة 1 الأول. وادفع commits صغيرة قابلة للمراجعة على فرع دورك وانت شغال. وحازم بيراجع وبيدمج في `develop` .

## الدور 1: حازم، Training Lead

**الفرع:** team/hazem-training-lead

### الدور ده يعني إيه

إنت صاحب جهد التدريب كله وfinal model. وإنت اللي بتتأكد إن dataset وmodels وevaluation كلهم ماشيين مع بعض، وإن النتيجة النهائية حقيقية. وبتراجع شغل كل عضو وبتدمجه. ومفيش حاجة بتوصل لـ `develop` أو `master` من غير مراجعتك.

### إنت مسؤول عن إيه

- training pipeline الكامل، من training tables النظيفة للtrained model.
- dataset، وده شامل تطبيق تصحيحات المراجعة لما المراجعة تخلص.
- تقرير model selection وtraining decisions اللي طالعة منه.
- مراجعة وfusion شغل الست فروع بتاعة الأدوار.
- retrain على الـ labels المراجعة في المرحلة 4.

### هتسلّم إيه

- training notebooks، متنفذة، بمخرجات محفوظة، على فرعك.
- final trained model وconfigه، مع تسجيل إصدار dataset وseed وhyperparameters بدقة.
- retrained model بعد المراجعة، مع تطبيق التصحيحات ونتائج إعادة evaluation.
- سجل fusion لكل تغيير مقبول: إيه هو، وإيه الاختبار أو evaluation اللي اتشغل، وإيه اللي غيّره.

### شغلك خطوة بخطوة

**المرحلة 1.** جهّز training pipeline. وتأكد إن training tables النظيفة بتتحمل صح وإن splits مطابقة لمجلدات الصوت. واكتب هيكل training notebook اللي كل دور هيستخدمه: تحميل البيانات، وshared config، ومعالجة seed، وصيغة مخرجات النتائج. وراجع شغل كل عضو في المرحلة 1 وهو واصل.

**المرحلة 2.** نسّق شغل models. لمى بتشغّل fine-tuning للنص، وسيرا بتشغّل audio model، وآية بتشغّل evaluation. وراجع validation results وأكّد الفائز حسب القاعدة في تقرير model selection: أعلى متوسط macro-F1 on validation، ولو الفرق بين EgyBERT وMARBERTv2 ضمن one standard deviation، اختار MARBERTv2. وسجّل القرار والأرقام اللي وراه.

**المرحلة 3.** أشرف على final test evaluation. وتأكد إن test set بتتقيّم مرة واحدة بالظبط، على frozen test set اللي فيها 584 صف، من غير re-split ومن غير ضبط على test set. وراجع error analysis. وتأكد إن integrated pipeline شغال من الأول للآخر.

**المرحلة 4.** بعد ما المراجعة تخلص، خد تعديلات المراجعة المصدرة من صفحة المراجعة. وطبّق التصحيحات على تسميات dataset. وrerun training notebooks على dataset المنظفة بarchitecture وhyperparameters نفسها. واطلب من آية إعادة evaluation. وسلّم final model.

### محتاج إيه من غيرك

- نتائج evaluation آية، عشان تختار الفائز وتأكد metrics النهائية.
- s model'Lama وسيرا المدرّبين وvalidation results بتاعتهم.
- منطق فرز ابتهال، عشان تتأكد إن الticket شغالة مع مخرجات real models.
- integrated pipeline من عرفة، عشان تتأكد إن النظام شغال من الأول للآخر.
- تعديلات المراجعة من الفريق، مصدرة من صفحة المراجعة، عشان تشغّل المرحلة 4.

### غيرك محتاج منك إيه

- هيكل training pipeline وshared config، بدري في المرحلة 1، عشان محدش يبني على إعداد مختلف.
- dataset، متحقق منها ومستقرة، عشان محدش يدرّب على بيانات غلط.
- المراجعات وfusion، بسرعة، عشان شغل محدش يفضل مركون من غير fusion.
- القرارات النهائية عن الفائز، وsplit freeze، وأسئلة الحقوق والموافقة.

### تعرف إزاي إنك خلصت

training notebooks بتشتغل من الأول للآخر من غير أخطاء. وfinal trained model على الـ labels المصححة، ومتقيّم مرة واحدة على frozen test set، وmetrics متسجلة مع إصدار dataset وconfig model والجهاز. وكل fusion متسجل. ومفيش ادعاء متقاسش.

## الدور 2: سيرا، Audio and Speech Engineer

**الفرع:** team/sira-audio-speech

**الحزم الزيادة:** torch وtransformers وlibrosa وsoundfile

## الدور ده يعني إيه

إنت صاحبة كل اللي جاي من الصوت نفسه: الصوت، والنص المحوّل، وaudio model. ومهمتك تثبتي الصوت ممكن يقولنا إيه زيادة عن النص، وتبني model اللي بيستخرج ده.

## إنت مسؤولة عن إيه

- audio pipeline: فحوصات الاستقبال، والتوحيد لـ WAV بتردد 16 كيلوهرتز وقناة واحدة، وقياس الجودة.
- transcription: Whisper large-v3، وconfig، و validation إن مخرجاته كويسة كفاية للتدريب عليها.
- direct audio classification model: wav2vec2 XLS-R 300M، وEgyptian checkpoint.
- audio features اللي بتغذي multi-task model، لو استخدمنا fusion.

## هتسلّمي إيه

- audio pipeline، مع فحوصات الاستقبال والتوحيد، متحقق منه على dataset كلها.
- STT validation: تقرير قصير بيوضح جودة Whisper transcription large-v3 على عينة، مع config المستخدمة.
- trained audio model، مع validation metrics لكل حقل.
- audio features، بصيغة موثقة، لو لمى استخدمتها للfusion.

## شغلك خطوة بخطوة

**المرحلة 1.** اتأكدي من الصوت. وتأكدي إن training audio files اللي عددها 5,751 هي WAV بتردد 16 كيلوهرتز وقناة واحدة ومطابقة لtraining tables. وابني فحص الاستقبال اللي بيرفض الصوت الوحش وبيسجل السبب. وشغّلي Whisper large-v3 على عينة وسجّلي transcription quality وconfig. وابني XLS-R baseline: حمّلي facebook/wav2vec2-xls-r-300m، وجمّدي encoder، ودرّبي رأس تصنيف بسيط على train split، وسجّلي validation metrics. وbaseline ده بيقول للفريق الصوت لوحده يقدر يعمل إيه.

**المرحلة 2.** درّبي audio model. ابدئي من IbrahimAmin/egyptian-arabic-Egyptian checkpoint wav2vec2-xlsr-53 لو اتحمّلت واتحقق منها. وخلّي encoder مجمد الأول، وبعدين اضبطي جزئيا الطبقات العليا من 2 لـ 4 طبقات. وسجّلي validation metrics لكل حقل ب5 seeds نفسها اللي بيستخدمها text model، عشان النتائج تتقارن. واختبري fusion الصوت مع النص بس لو audio baseline أظهر إشارة حقيقية: فلو audio model مضافش حاجة فوق النص لوحده، قولي ده بوضوح ومتفرضيش fusion. ولو fusion ساعد، سلّمي audio features للمى بالصيغة الموثقة.

**المرحلة 3.** ادعمي evaluation. وادي آية audio final model وconfigه عشان تقدر تقيّمه على test set. وساعدي في error analysis في الأماكن اللي الصوت فيها بيفسر غلطة عملها text model.

**المرحلة 4.** مفيش شغل صوتي جديد إلا لو تصحيحات المراجعة غيّرت مجموعة الصوت. ولو الـ labels المصححة غيّرت أي صف صوتي، rerun affected validation وسجّلي التغيير.

### محتاجة إيه من غيرك

- dataset وtraining tables، من حازم، متحقق منها.
- transcription وبيانات الثقة الوصفية، وهي موجودة أصلا في dataset.
- نتائج s text model'Lama، عشان تقارني الصوت بالنص وتقرري fusion.
- مجموعة evaluation آية، عشان تشغّلي validation بنفس الطريقة اللي الكل بيستخدمها.

### غيرك محتاج منك إيه

- audio pipeline وفحوصات الاستقبال، عشان الصوت الوحش ميوصلش للتدريب من غير ما حد ياخد باله.
- STT validation، عشان الفريق يعرف إن النصوص موثوقة.
- audio model وvalidation metrics بتاعته، عشان آية تقدر تقيّمه ولمى تقرر fusion.
- audio features، بالصيغة الموثقة، لو استخدمنا fusion.

### تعرفي إزاي إنك خلصتي

audio pipeline بيتحقق من كل training files اللي عددها 5,751. وSTT validation متسجل. وaudio model عنده validation metrics لكل حقل ب5 seeds. وقرار fusion متسجل بالأدلة، سواء كانت الإجابة آه أو لأ.

## الدور 3: لمى، Text Classification Engineer

**الفرع:** team/lama-text-classification

**الحزم الزيادة:** torch وtransformers وdatasets

### الدور ده يعني إيه

إنت صاحبة base model: text classifier اللي بيقرأ نص وبيتوقع الحقول الأربعة. وده model اللي النظام كله معتمد عليه. ومهمتك تدرّبيه كويس، وتثبتيه with validation، وتسلّمي model جاهز للevaluation والربط.

### إنت مسؤولة عن إيه

- fine-tuning: text classifier لـ EgyBERT وMARBERTv2.

- multi-task setup: shared encoder واحد بأربع رؤوس، رأس لكل حقل.
- إعداد hyperparameters: fine-tuning، وseeds، وclass weights.
- validation results اللي بتحدد الفائز.

## هتسلّمي إيه

- text fine-tuned models، مع config وvalidation metrics لكل حقل.
- multi-task model، بshared encoder والـ four heads.
- نتائج fine-tuning: متوسط macro-F1 on validation لكل حقل، mean وstandard deviation عبر 5 seeds، لكل من EgyBERT وMARBERTv2.
- توصية الفائز، بالأرقام اللي وراها، مع اتباع قاعدة كسر التعادل.

## شغلك خطوة بخطوة

**المرحلة 1.** جهّزي single-task fine-tuning. وحمّلي training tables النظيفة. وحمّلي الـ tokenizer لـ EgyBERT وMARBERTv2. وتأكدي إن الحد الأقصى لطول التسلسل 128 بيغطي البيانات: أطول نص هو 72 كلمة، يعني حوالي 93 رمز، فـ 128 أكتر من كفاية. وابني training script اللي بيضبط one model على حقل واحد. وشغّلي smoke test على مجموعة فرعية صغيرة عشان تتأكدي إن pipeline شغال من الأول للآخر. واعتمدي training infrastructure من آية أول ما توصل: experiment tracking، وconfig format، وseed handling. ومتشغّليش fine-tuning الكامل على إعداد مختلف.

**المرحلة 2.** شغّلي single-task fine-tuning. ودرّبي EgyBERT وMARBERTv2 بالتوازي، one model لكل حقل، ب5 seeds (42 و1337 و2023 و7 و99). واستخدمي hyperparameters من تقرير model selection: learning rate 2e-5، مع التجربة على 1e-5 و2e-5 و3e-5؛ وeffective batch size 16؛ وepochs 3، بحد أقصى 5، مع early stopping بصبر 2 على متوسط macro-F1 للvalidation؛ وwarmup ratio 0.1؛ وweight decay 0.01، مع ضبطه صراحة؛ وAdamW؛ وdropout بمقدار 0.1؛ وgradient clipping بحد أقصى 1.0؛ وmixed precision مفعّلة. وطبّقي class weights من dataset، مع تحديد وزن churn بحد أقصى بين 10 و20. وسجّلي متوسط macro-F1 on validation لكل حقل، mean وstandard deviation عبر seeds، لكل من modelين. وبعدين ابني multi-task model: shared encoder واحد، وأربع رؤوس، مع uncertainty weighting Kendall، وclass weights المحفوظة جوه كل رأس. ودرّبيه بنفس الطريقة وسجّلي نفس metrics. وقارني single-task بmulti-task on validation وسجّلي مين أحسن لكل حقل.

**المرحلة 3.** سلّمي الفائز. وادي آية winning model وexact config عشان تقدر تقيّمه مرة واحدة على test set. ومتضبطيش أي حاجة على test set. وساعدي في error analysis في الأماكن اللي model فيها بيخلط بين فئتين.

**المرحلة 4.** بعد تصحيحات المراجعة، rerun winning setup على dataset المنظفة. ومفيش تغيير في architecture ولا في hyperparameters. وسجّلي validation results والاختبار الجديدة.

### محتاجة إيه من غيرك

- dataset وtraining tables، من حازم، متحقق منها.
- training infrastructure من آية، بدري في المرحلة 1: experiment tracking، وconfig format، وseed handling.
- class weights، وهي موجودة أصلا في dataset.
- audio features من سيرا، بس لو استخدمنا fusion في المرحلة 2.

### غيرك محتاج منك إيه

- fine-tuned models وvalidation metrics بتاعتها، عشان آية تقدر تقيّم الفائز وحازم يأكد القرار.
- توصية الفائز بالأرقام اللي وراها.
- exact config للwinning model، عشان آية تقدر تعيد إنتاج test evaluation وحازم يعيد تشغيله في المرحلة 4.

### تعرفي إزاي إنك خلصتي

modelين مضبوطين على الحقول الأربعة ب5 seeds. ومتوسط macro-F1 on validation لكل حقل متسجل mean وstandard deviation. وmulti-task trained model ومتقارن. والفائز متسمّي بالأرقام اللي وراه، مع اتباع قاعدة كسر التعادل. وconfig الفائز موثقة وقابلة لإعادة الإنتاج.

## الدور 4: آية، ML Infrastructure and Evaluation Engineer

**الفرع:** team/aya-ml-infrastructure-evaluation

**الحزم الزيادة:** scikit-learn وpandas وnumpy

### الدور ده يعني إيه

إنت صاحبة الآلية اللي بتخلي التدريب موثوق: infrastructure اللي الكل بيشتغل عليها، وevaluation اللي بيثبت models شغالة ولا لأ. وإنت اللي بتتأكدي إن النتيجة حقيقية وقابلة لإعادة الإنتاج ومتقاسة بنفس الطريقة للكل. ولو رقم مينفعش يتعاد إنتاجه، فهو مبيتحسبش، وإنت اللي بتكشفي ده.

## إنت مسؤولة عن إيه

- training infrastructure: experiment tracking، وعمليات hyperparameter sweep، وconfig format، وseed handling.
- TF-IDF baseline، وهو المرجع اللي كل models بتتقارن بيه.
- evaluation harness: الكود اللي بيشغّل model على split وبيبلغ عن metrics.
- metrics، وerror analysis، وevaluation reports.
- القاعدة اللي بتقول إن test set بتتقيّم مرة واحدة بالظبط.

## هتسلّمي إيه

- training infrastructure، موثقة، مع config format وseed handling، متسلمة بدري في المرحلة 1.
- TF-IDF baseline، مع validation metrics لكل حقل.
- evaluation harness، مع metrics اللي بتبلغ عنها وصيغة مخرجاتها.
- evaluation reports: validation results لكل model، وfinal test evaluation، وerror analysis.
- run records لكل تجربة: رقم التشغيل، والفرضية، والعامل الوحيد المتغير، والظروف الثابتة، وmetrics، والمخرجات.

## شغلك خطوة بخطوة

**المرحلة 1.** ابني infrastructure الأول، قبل ما حد يشغّل التدريب الكامل. وحدّدي model: config format، وhyperparameters، وseed، وإصدار dataset، وsplit. وحدّدي seed handling: 5 seeds (42 و1337 و2023 و7 و99)، وكل تجربة بتشغّل 5 seeds كلها وبتبلغ عن mean وstandard deviation. وحدّدي run record format: رقم التشغيل، والفرضية، والعامل الوحيد المتغير، والظروف الثابتة، وmetrics، والمخرجات. وابني evaluation harness: بتاخد trained model وsplit، وبتشغّله، وبتبلغ عن macro-F1 لكل حقل، وaccuracy، وconfusion matrix، بصيغة مخرجات ثابتة. وابنيها على fixtures الأول، بmock predictions، عشان تشتغل قبل ما يكون فيه أي real model. وبعدين شغّلي TF-IDF baseline: ميزات TF-IDF مع logistic regression، one model لكل حقل، على train split، مع evaluation على validation split. وسجّلي validation metrics. وbaseline ده هو المرجع اللي لازم neural models تتفوق عليه.

**المرحلة 2.** ادعمي fine-tuning. وساعدي لمى وسيرا يشغّلوا تجاربهم على infrastructure. وشغّلي hyperparameter sweep لtext model مع لمى: learning rate 1e-5 و2e-5 و3e-5، وسجّلي validation result لكل واحد. وشغّلي validation لكل model وسجّلي metrics بنفس الطريقة كل مرة. ومتخليش حد يبلغ عن نتيجة من غير سجل تشغيل.

**المرحلة 3.** شغّلي final test evaluation. وخدي winning model من لمى وaudio model من سيرا. وقيّمي كل واحد مرة واحدة بالظبط على frozen test set اللي فيها 584 صف. وسجّلي metrics كل حقل، وعدد العينات، والإخفاقات. وبعدين شغّلي error analysis :model بيفشل فين، وإيه المشترك بين الإخفاقات، وإيه اللي هيصلحها. واكتبي تقرير evaluation بالأرقام الفعلية، وconfig، والجهاز، والقيود. وقولي بوضوح إيه اللي البيانات متقدرش تدعمه.

**المرحلة 4.** بعد تصحيحات المراجعة، re-evaluate retrained model على test set، مرة واحدة، بنفس الطريقة. وسجّلي النتائج الجديدة جنب القديمة عشان التغيير يكون واضح.

### محتاجة إيه من غيرك

- dataset وtraining tables، من حازم، متحقق منها.
- trained models من لمى وسيرا، مع configها الدقيقة، عشان تقيّميها.
- منطق الفرز من ابتهال، عشان تقيّمي اتفاق التوجيه.
- integrated pipeline من عرفة، عشان تقيسي latency من الأول للآخر.

### غيرك محتاج منك إيه

- training infrastructure، بدري في المرحلة 1، عشان لمى وسيرا يشتغلوا على نفس الإعداد.
- evaluation harness، عشان الكل يقيس بنفس الطريقة.
- TF-IDF baseline، عشان كل model يكون ليه مرجع يتفوق عليه.
- evaluation reports، عشان حازم يقدر يأكد الفائز والفريق يشوف الأرقام الحقيقية.
- run records، عشان كل تجربة تكون قابلة للتتبع.

### تعرفي إزاي إنك خلصتي

infrastructure موثقة ولمى وسيرا بيستخدموها. وTF-IDF baseline متسجل. وevaluation harness شغالة على البيانات التجريبية وعلى real models. وfinal test evaluation متسجل بexact config والجهاز. وerror analysis مكتوب. وكل تجربة ليها سجل تشغيل.

## الدور 5: ابتهال، Triage and Routing Engineer

**الفرع:** team/ibthal-triage-routing

**الحزم الزيادة:** pandas وnumpy

## الدور ده يعني إيه

إنت صاحبة القرار: إزاي مخرجات four models بتتحول لqueue priority وcallback ticket يقدر الموظف يشتغل عليها. ومهمتك تخلي قرار النظام قابل للتفسير. والموظف لازم يقدر يقرأ الticket ويعرف ليه العميل ده بيتتصل بيه الأول، وإيه اللي العميل محتاجه.

## إنت مسؤولة عن إيه

- queue priority logic: إزاي فئة الشكوى ودرجة الاستعجال وحالة العميل واحتمال فقدان العميل بيتحولوا لpriority رد.
- ticket format :callback ticket generation، والحقول اللي بتحملها، والسبب اللي بتعرضه.
- routing agreement evaluation: مدى تطابق توجيه النظام مع ideal راجعه إنسان.
- القاعدة اللي بتقول إن الاستعجال بييجي من الشكوى بس، ومش من النبرة أبدا، وإنه مفيش خلط بين الحالة والاستعجال أبدا.

## هتسلّمي إيه

- قواعد الفرز، مكتوبة، مع priority اللي كل مزيج من الحقول بينتجها.
- ticket format، مع كل حقل بتحمله وإيه معنى كل حقل.
- ticket generator، اللي بياخد مخرجات four models وبينتج ticket.
- evaluation التوجيه: الاتفاق مع ideal اللي راجعه إنسان، والحالات اللي النظام بيختلف فيها.

## شغلك خطوة بخطوة

**المرحلة 1.** اكتبي قواعد الفرز. ابدئي من تعريفات الحقول: فئة الشكوى ليها 11 قيمة، ودرجة الاستعجال ليها 3، وحالة العميل ليها 5، واحتمال فقدان العميل ليه قيمتين. وحدّدي priority: الاستعجال العالي بييجي الأول، واحتمال الفقدان بييجي الأول، ومزيج الاتنين بييجي الأول قبل كل حاجة. واكتبي القواعد كجدول، كل مزيج من الاستعجال واحتمال الفقدان مربوط بpriority، عشان ميكونش فيه غموض. وحدّدي ticket format: مرجع العميل، والحقول الأربعة، وpriority، وسبب التوجيه بكلمات واضحة، وسطر الافتتاح المقترح، وإصدار القاعدة المساهمة. وابني ticket generator على fixtures: زوّديه بmock model outputs وتأكدي إن الticket اللي بينتجها مطابقة لجدول القواعد.

**المرحلة 2.** اختبري القواعد على مخرجات validation. وخدي توقعات validation من لمى وشغّليها في ticket generator. وتأكدي إن الأولويات شكلها صح وإنه مفيش ticket من غير سبب. وسجّلي الحالات اللي القواعد فيها بتنتج priority مفاجئة وعدّلي القواعد لو التعريفات بتدعم ده. ومتغيّريش تعريفات الحقول؛ غيّري بس الربط بين الحقول وpriority.

**المرحلة 3.** خلّصي الفرز على مخرجات real models. وخدي test predictions النهائية من آية وشغّليها في ticket generator. وقيسي اتفاق التوجيه مع ideal اللي راجعه إنسان: والهدف هو 80 بالمئة. وسجّلي الاتفاق والاختلافات، وإيه كان كل اختلاف. وسلّمي ticket generator لعرفة عشان يربطه.

**المرحلة 4.** لو تصحيحات المراجعة غيّرت أي label، rerun evaluation التوجيه على البيانات المصححة وسجّلي التغيير.

### محتاجة إيه من غيرك

- تعريفات الحقول و classification، من حازم، وهي متفق عليها أصلا.
- مخرجات models، من لمى وسيرا، الأول كتوقعات validation في المرحلة 2، وبعدين كتوقعات اختبار في المرحلة 3.
- هدف اتفاق التوجيه وideal اللي راجعه إنسان، من الفريق.
- مجموعة evaluation آية، عشان تقيسي اتفاق التوجيه بنفس الطريقة اللي بيتقاس بيها كل حاجة تانية.

### غيرك محتاج منك إيه

- قواعد الفرز، مكتوبة، عشان الفريق يقدر يراجعها قبل ما تتنفذ.
- ticket format، عشان عرفة يقدر يبني demo حواليها والموظف يقدر يقراها.
- ticket generator، عشان عرفة يقدر يربطه في pipeline.
- evaluation التوجيه، عشان الفريق يعرف التوجيه شغال ولا لأ.

### تعرفي إزاي إنك خلصتي

قواعد الفرز مكتوبة كجدول كامل من غير أي مزيج ناقص. وticket generator بينتج ticket صح لكل مدخل وهمي. واتفاق التوجيه متقاس على test predictions الحقيقية ومتسجل مع الاختلافات. وticket format موثقة ومستخدمة في demo.

## الدور 6: عرفة، Systems Integration and Demo Engineer

**الفرع:** team/arafa-systems-demo

**الحزم الزيادة:** fastapi وuvicorn وgradio

### الدور ده يعني إيه

إنت صاحب النظام النهائي: pipeline اللي بيشتغل من الأول للآخر، وdemo اللي بيوضح شغله. ومهمتك تاخد الأجزاء اللي غيرك ببنيها وتخليها تشتغل مع بعض كنظام واحد، وبعدين تثبت إنه شغال بشكل نظيف وبسرعة كافية.

### إنت مسؤول عن إيه

- الربط الكامل من الأول للآخر: رسالة صوتية بتدخل، وticket بتخرج، مع ربط كل خطوة.
- واجهة البرمجة API وdemo: الواجهة اللي الفريق وحكام المسابقة بيستخدموها.
- قياس latency: p95 أقل من 10 ثواني للمقاطع لحد 60 ثانية.
- مخرجات المسابقة اللي بتعرض النظام وهو شغال.
- التشغيل المحلي القابل لإعادة الإنتاج: أي عضو في الفريق يقدر يشغّل النظام بنفس الطريقة.

### هتسلّم إيه

- integrated pipeline، مع ربط كل خطوة وتدفق البيانات خلاله.
- demo، شغال بشكل نظيف، مع الرفع والتسجيل من المتصفح.
- واجهة البرمجة API، مع الـ endpoints اللي demo بيستخدمها.
- latency report: p95 latency للمقاطع لحد 60 ثانية، متقاس بشكل منفصل لكل من CPU وGPU أو إعداد المزود.
- تعليمات التشغيل، عشان أي حد يقدر يعيد إنتاج demo.

### شغلك خطوة بخطوة

**المرحلة 1.** ابني الهيكل. وحدّد pipeline stages: الاستقبال، وtranscription، و classification، والفرز، والticket. وابني كل مرحلة كوحدة منفصلة بمدخل ومخرج واضحين. وابني الهيكل بmocks الأول: مرحلة transcription بترجع نص ثابت، وclassification stage بترجع حقول ثابتة، ومرحلة الفرز بترجع ticket ثابتة. وتأكد إن البيانات بتتدفق من المرحلة الأولى للأخيرة. وابني demo skeleton: رفع، وتشغيل، وعرض ticket. وحافظ على استقرار عقود واجهة البرمجة API والبيانات عشان الأدوار التانية تقدر تبني عليها.

**المرحلة 2.** استبدل mocks بالمكونات الحقيقية لما توصل. ولما audio pipeline من سيرا يكون جاهز، اربطه. ولما text model من لمى يكون جاهز، اربطه. ولما ticket generator من ابتهال يكون جاهز، اربطه. واختبر كل استبدال لوحده قبل ما تكمل. وسجّل إيه من المكونات حقيقي وإيه لسه وهمي، عشان الفريق يعرف دايما pipeline state.

**المرحلة 3.** اربط وقيس. ومع وجود كل المكونات الحقيقية، شغّل pipeline الكامل على validation set وتأكد إن tickets صح. وقيس latency: شغّل pipeline على مقاطع لحد 60 ثانية وسجّل p95 latency، بشكل منفصل لـ CPU وGPU لو الاتنين مستخدمين. والهدف هو p95 أقل من 10 ثواني. ولو latency فوق الهدف، دوّر على أبطأ مرحلة وبلّغ صاحب المرحلة دي. وابني final demo: الرفع، والتسجيل من المتصفح، وقائمة الانتظار الحية، وأدلة الticket، وموجز الرد. وشغّل demo خمس مرات ورا بعض بشكل نظيف. واكتب تعليمات التشغيل عشان أي عضو في الفريق يقدر يعيد إنتاجه.

**المرحلة 4.** بعد retrain، حدّث demo بـretrained model وأعد قياس latency. وتأكد إن demo لسه شغال بشكل نظيف.

### محتاج إيه من غيرك

- audio pipeline من سيرا، عشان تربطه في مرحلة الاستقبال.
- text model من لمى، عشان تربطه في classification stage.
- ticket generator من ابتهال، عشان تربطه في مرحلة الفرز.
- مجموعة evaluation آية، عشان تقيس latency بنفس الطريقة اللي بيتقاس بيها كل حاجة تانية.
- عقود واجهة البرمجة API والبيانات، المتفق عليها مع الفريق، عشان المكونات تتناسب مع بعض.

### غيرك محتاج منك إيه

- pipeline skeleton وعقود واجهة البرمجة API، بدري في المرحلة 1، عشان الكل يبني مكونات بتتناسب.
- integrated pipeline، عشان الفريق يشوف النظام شغال من الأول للآخر.
- demo، عشان مخرجات المسابقة تكون حقيقية.
- latency report، عشان الفريق يعرف النظام سريع كفاية ولا لأ.
- تعليمات التشغيل، عشان أي حد يقدر يعيد إنتاج demo.

### تعرف إزاي إنك خلصت

pipeline شغال من الأول للآخر بكل المكونات الحقيقية. وdemo شغال بشكل نظيف خمس مرات ورا بعض. وlatency report بيوضح p95 أقل من 10 ثواني للمقاطع لحد 60 ثانية، مع تسجيل الجهاز. وتعليمات التشغيل بتخلي أي عضو في الفريق يعيد إنتاج demo.

## إزاي بنشتغل بالتوازي

المراحل متصممة عشان محدش يستنى في المرحلة 1، والانتظار في المراحل اللي بعدها هو الترتيب الطبيعي للشغل، مش توقف. والجدول اللي تحت بيوضح مين بيعمل إيه في كل مرحلة.

| المرحلة | حازم | سيرا | لمى | آية | ابتهال | عرفة |
|---|---|---|---|---|---|---|
| **المرحلة 1: الأساسيات** | هيكل training pipeline، وshared config | audio pipeline، وSTT validation، وbaseline XLS-R | single-task fine-tuning setup، وsmoke test، واعتماد s'Aya infrastructure | training infrastructure، وevaluation harness، وTF-IDF baseline | جدول قواعد الفرز، وticket format، والمولّد على fixtures | pipeline skeleton على mocks، وdemo skeleton، وعقود API |
| **المرحلة 2: core models** | التنسيق، وvalidation review، وتأكيد الفائز | audio model، وfusion test | single-task fine-tuning، وmulti-task model | hyperparameter sweep، وvalidation، وrun records | اختبار القواعد على مخرجات validation | استبدال mocks بالمكونات الحقيقية لما توصل |
| **المرحلة 3: evaluation والربط** | الإشراف على test evaluation، ومراجعة error analysis | دعم evaluation، وaudio error analysis | تسليم الفائز، ودعم error analysis | final test evaluation مرة واحدة، وerror analysis، والتقرير | الانتهاء من الفرز على المخرجات الحقيقية، واتفاق التوجيه | الربط الكامل، وقياس latency، وfinal demo |
| **المرحلة 4: retrain** | تطبيق تصحيحات المراجعة، وrerun training | إعادة تشغيل validation لو اتغيرت صفوف صوتية | إعادة تشغيل winning setup على البيانات المنظفة | إعادة evaluation retrained model، وتسجيل التغيير | إعادة تشغيل evaluation التوجيه على البيانات المصححة | تحديث demo بretrained model، وإعادة قياس latency |

مراجعة الـ labels شغالة كل يوم بالتوازي مع المراحل الأربعة كلها. وكل عضو بيراجع الـ labels على صفحة المراجعة الحية يوميا. والمراجعة مبتعطلش التدريب.

## الاعتماديات ونقط الانتظار

القسم ده بيسرد كل مكان فيه عضو محتاج مخرجات عضو تاني، وإيه اللي يتعمل في الحالة دي. اقراه قبل ما تبدأ مرحلة عشان تعرف إنت مستني إيه وإيه اللي تقدر تعمله في الوقت ده.

### نقطة الانتظار 1: لمى بتستنى training infrastructure من آية

fine-tuning الكامل بتاع لمى بيشتغل على s infrastructure: experiment tracking'Aya، وconfig format، وseed handling. وآية بتسلّم infrastructure بدري في المرحلة 1، قبل baseline. ولمى مبتقعدش من غير شغل: فهي بتبني single-task fine-tuning setup، وتحميل البيانات، والـ tokenizer، وتحميل model في المرحلة 1، وبتشغّل smoke test على مجموعة فرعية صغيرة بconfig بسيطة. وأول ما s infrastructure'Aya توصل، لمى بتعتمدها وبتشغّل fine-tuning الكامل. ولو s infrastructure'Aya اتأخرت، لمى بتقول لحازم فورا عشان الخطة تتعدل.

### نقطة الانتظار 2: ابتهال بتستنى مخرجات real models عشان تخلّص الفرز

ابتهال بتكتب قواعد الفرز وticket format في المرحلة 1 باستخدام تعريفات الحقول وmock model outputs. ومش محتاجة أي model عشان ده. وبتخلّص ابتهال القواعد في المرحلة 3 على test predictions الحقيقية من لمى وسيرا. وده الترتيب الطبيعي: القواعد بتتكتب الأول، وبعدين بتتفحص مقابل الواقع. وابتهال مبتستناش في المرحلة 1 ولا في المرحلة 2.

### نقطة الانتظار 3: عرفة بيستنى المكونات الحقيقية عشان يربط

عرفة بيبني pipeline skeleton وdemo skeleton في المرحلة 1 باستخدام mocks. ومش محتاج أي مكون عشان ده. وبيستبدل عرفة mocks بالمكونات الحقيقية لما توصل في المرحلة 2 والمرحلة 3. والربط النهائي بيستنى final models في المرحلة 3. وده الترتيب الطبيعي: الهيكل بيتبني الأول، وبعدين الأجزاء الحقيقية بتتلائم فيه. وعرفة مبيستناش في المرحلة 1.

### نقطة الانتظار 4: آية بتستنى models عشان تقيّمها

آية بتبني evaluation harness في المرحلة 1 على fixtures بـmock predictions. ومش محتاجة أي model عشان ده. وبتشغّل آية TF-IDF baseline في المرحلة 1، وده مش محتاج neural model. وبتقيّم آية real models في المرحلة 3، لما لمى وسيرا يسلّموها. وده الترتيب الطبيعي: evaluation harness بتتبني الأول، وبعدين real models بتتقاس بيها. وآية مبتستناش في المرحلة 1 ولا في المرحلة 2.

### نقطة الانتظار 5: fusion بيستنى audio features من سيرا

لو اتسعى لـfusion الصوت مع النص في المرحلة 2، فلمى بتحتاج audio features من سيرا. وسيرا بتسلّم audio features في المرحلة 1 والمرحلة 2. وmulti-task model بتاع لمى بيبدأ بالنص بس وبيضيف audio features لما تجهز. وfusion اختياري وبيقرره الدليل: فلو audio baseline مأظهرش إشارة حقيقية فوق النص لوحده، فمفيش fusion ومفيش انتظار. ومحدش بيتعطل بسبب fusion.

### نقطة الانتظار 6: المرحلة 4 بتستنى المراجعة

retrain مبتقدرش تبدأ لحد ما المراجعة تخلص، بعد حوالي شهر من بدايتها. وده الانتظار الصعب الوحيد، وهو مقصود. والمراحل 1 و2 و3 بتمشي بالتوازي مع المراجعة ومبتستناهاش. والفريق بيدرّب على الـ labels الحالية، ولما المراجعة تخلص، حازم بيطبّق التصحيحات والفريق بيعيد تشغيل training notebooks. ومفيش حاجة لتغيير architecture ولا hyperparameters لـretrain، فهي إعادة تشغيل، مش إعادة بناء.

### قاعدة الانتظار

لو إنت مستني مخرجات عضو تاني، متوقفش. وابني على fixtures أو mocks، واكتب الاختبارات، ووثّق الواجهة، وحضّر الربط. ولما المخرجات الحقيقية توصل، بدّلها واختبر. ولو الانتظار طال أكتر من المتوقع، قول لحازم. ومحدش بيستنى بصمت.

## قواعد الشغل

القواعد دي بتنطبق على الكل، في كل مرحلة.

- **الفروع.** اشتغل بس على فرع دورك. ومتعملش commit مباشرة على `develop` أو `master` أبدا. وحازم بيدمج في `develop` ، وبيدمج النتيجة النهائية المتحقق منها في `master` .
- **الالتزامات.** ادفع commits صغيرة قابلة للمراجعة وانت شغال. وكل رسالة commit بتقول إيه اللي اتغيّر وليه. والـ commit اللي بيغيّر dataset أو مخطط أو واجهة بيقول ده في الرسالة.
- **المراجعات.** حازم بيراجع كل batch قبل ما يدمجها. والمراجعة بتسأل عن الأدلة: إيه الاختبار اللي اتشغل، وإيه نتيجته، وإيه اللي اتغيّر. ونجاح اختبار لوحده ميثبتش إن التغيير صح.
- **التجارب.** كل تجربة ليها سجل تشغيل: رقم التشغيل، والفرضية، والعامل الوحيد المتغير، والظروف الثابتة، وmetrics، والمخرجات. وغيّر عامل تجريبي واحد في كل مرة. ومتستنتجش فائز من split مختلف أو من كذا تغيير متزامن.
- **test set.** test set بتتقيّم مرة واحدة بالظبط، بعد تجميد validation. ومفيش ضبط على test set. ومفيش re-split بعد ما النتائج تظهر.
- **الـ labels.** الـ labels جاية من model وبيتم مراجعتها. ومتقدمش label أبدا على إنها متحقق منها بشريا لحد ما المراجعة تعتمدها. وصفحة المراجعة هي المكان الوحيد اللي الـ labels فيه بتتعتمد أو بتتعدل.
- **الاجتماعات.** اجتماع أسبوعي واحد للربط واتخاذ القرارات. وكل عضو بيبلغ عن المخرجات المنجزة، والأدلة والاختبارات، وتغييرات التجارب ونتائجها، والعوائق، والاعتمادية اللي بعدها.
- **الأسرار.** متحطش أسرار أو كلمات مرور أو رموز أو تسجيلات خاصة في المستودع أو في commit أو في رسالة دردشة أبدا. واستخدم fixtures اصطناعية للتطوير الروتيني.
- **اللغة.** المستندات وتعليقات الكود بالإنجليزية. والمصطلحات التقنية بتفضل بالإنجليزية.

## خط النهاية

المشروع بيكون خلصان لما كل النقط اللي تحت دي تتحقق:

1. text model وaudio trained modelين ومتحقق منهم ومتقيّمين مرة واحدة على frozen test set، مع تسجيل metrics.
2. منطق الفرز بينتج ticket صح لكل توقع اختبار، مع قياس اتفاق التوجيه.
3. pipeline شغال من الأول للآخر بكل المكونات الحقيقية، وdemo شغال بشكل نظيف خمس مرات ورا بعض.
4. latency report بيوضح p95 أقل من 10 ثواني للمقاطع لحد 60 ثانية، مع تسجيل الجهاز.
5. المراجعة خلصت، والتصحيحات اتطبقت، وmodels اتعاد تدريبها واتعاد evaluationها على الـ labels المصححة.

6. كل تجربة ليها سجل تشغيل. وكل fusion متسجل. ومفيش ادعاء مقدم متقاسش.
7. مخرجات المسابقة كاملة: demo، وتعليمات التشغيل، وملخص النتائج بالأرقام الفعلية.

ولما نوصل لخط النهاية، حازم بيدمج النتيجة النهائية المتحقق منها في `master` .